Method	HTTP verb	Expected operation
`MapGet(path, handler)`	`GET`	Fetch data only; no modification of state. May be safe to cache.
`MapPost(path, handler)`	`POST`	Create a new resource.
`MapPut(path, handler)`	`PUT`	Create or replace an existing resource.
`MapDelete(path, handler)`	`DELETE`	Delete the given resource.
`MapPatch(path, handler)`	`PATCH`	Modify the given resource.
`MapMethods(path, methods, handler)`	Multiple verbs	Multiple operations.
`Map(path, handler)`	All verbs	Multiple operations.
`MapFallback(handler)`	All verbs	Useful for SPA fallback routes.

RESTful applications (as described in chapter 2) typically stick close to these verb uses where possible, but some of the actual implementations can differ, and people can easily get caught up in pedantry. Generally, if you stick to the expected operations described in table 5.1, you'll create a more understandable interface for consumers of the API.

NOTE You may notice that if you use the `MapMethods()` and `Map()` methods listed in table 5.1, your API probably doesn't correspond to the expected operations of the HTTP verbs it supports, so I avoid these methods where possible. `MapFallback()` doesn't have a path and is called *only* if no other endpoint matches. Fallback routes can be useful when you have a SPA that uses client-side routing. See http://mng.bz/9DMl for a description of the problem and an alternative solution.

As I mentioned at the start of section 5.2.2, testing APIs that use verbs other than `GET` is tricky in the browser. You need to use a tool that allows sending arbitrary requests such as Postman (https://www.postman.com) or the HTTP Client plugin in JetBrains Rider. In chapter 11 you'll learn how to use a tool called Swagger UI to visualize and test your APIs.